UCS645

PARALLEL & DISTRIBUTED COMPUTING

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 6:

Submitted by: Sujal Mallick (102303266)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Device Queries

1. Objective

To understand GPU architecture and parallel programming using CUDA. The assignment focuses on querying device properties, implementing parallel reduction for array sum, and performing matrix addition on the GPU. Performance and memory behavior of CUDA programs are analyzed

2. Problem Description

Device Query – Retrieve GPU properties such as compute capability, memory sizes, warp size, and execution limits.

4. Results

GPU: NVIDIA GeForce RTX 3050 Laptop GPU

Compute Capability: 8.6

Max Threads/Block: 1024

Block Dim: 1024 1024 64

Grid Dim: 2147483647 65535 65535

Global Memory: 4294508544

Shared Memory: 49152

Constant Memory: 65536

Warp Size: 32

5. Discussion

1. The compute capability defines the GPU architecture and supported features.
2. Maximum block and grid dimensions determine how kernels are launched.
3. Warp size (32) indicates the number of threads executed together.
4. Shared memory is fast but limited, while global memory is large but slower.
5. Double precision support depends on compute capability.

Array Sum

1. Objective
To compute the sum of array elements using parallel CUDA threads.

2. Problem Description
An array of floating-point numbers is generated on the host. The sum is computed on the GPU using parallel execution.

3. Methodology

1. Generated input array on host
2. Allocated device memory (cudaMalloc)
3. Copied data from host to device
4. Configured kernel launch:
   Block size = 256
   Grid size = ceil(N / block size)
5. Used atomicAdd() to accumulate sum
6. Copied result back to host
7. Freed device memory

4. Results
Computed Sum = Correct value (e.g., 1024 for all 1s)

Output verified correct

5. Discussion

1. Parallel execution reduces computation time
2. Atomic operations introduce synchronization overhead
3. Performance depends on:
4. Number of threads
5. Memory access pattern
6. Not fully optimized due to atomic contention

6. Conclusion
Parallel execution reduces computation time
Atomic operations introduce synchronization overhead
Performance depends on:
Number of threads
Memory access pattern

Matrix Addition

1. Objective
To perform matrix addition using CUDA and analyze computational and memory operations.

2. Problem Description
Two matrices $A$and $B$are added element-wise to produce matrix C:

$$C[i][j] = A[i][j] + B[i][j]$$

3. Methodology

1. Initialized matrices on host
2. Allocated device memory
3. Copied matrices to GPU
4. Used 2D grid and block configuration
5. Each thread computed one element
6. Copied result back to host

4.Results
Matrix addition executed successfully

5.Performance Discussion

1. Floating Point Operations

Each element performs **1 addition**

$$Total = N \times N$$

2. Global Memory Reads

Each thread reads:

- 1 element from A
- 1 element from B

$$TotalReads = 2 \times N^2$$

3. Global Memory Writes

Each thread writes:

$$TotalWrites = N^2$$

6. Conclusion

CUDA efficiently performs matrix addition using parallel threads. The operation is limited by memory bandwidth rather than computation. Proper memory access patterns can further improve performance.